

Cryptanalytic Extraction of Deep Neural Networks with Non-Linear Activations

Roderick Asselineau^{1,2,3}, Patrick Derbez¹, Pierre-Alain Fouque^{1,4}, Brice Minaud³

¹ Airbus, France

`roderick.asselineau@airbus.com`

² Univ Rennes, Inria, CNRS, IRISA, Rennes, France

`{patrick.derbez,pierre-alain.fouque}@inria.fr`

³ Inria, École Normale Supérieure - PSL, Paris, France

`brice.minaud@inria.fr`

⁴ Institut universitaire de France (IUF), France

Abstract. Deep neural networks (DNNs) are today’s central machine learning engines, yet their parameters represent valuable intellectual property exposed to extraction through black-box queries. While existing cryptanalytic attacks have primarily targeted ReLU-based architectures, this work extends model-stealing techniques to a broad class of non-linear activation functions, including GELU, SiLU, SELU, Sigmoid, and others. We present the first universal black-box attack capable of recovering both weights and biases from networks whose activations converge to linear behavior outside narrow non-linear regions. Our method generalizes prior geometric approaches by leveraging higher-order derivatives and adjacent linear zone analysis, bypassing the need for non-differentiability. We show that, for several activations, neuron signatures can be recovered more easily than in the ReLU case, and we further demonstrate that activation functions themselves can be identified when not publicly known. Our results broaden the scope of cryptanalytic model extraction, revealing that the secrecy of activation functions or smoothness of nonlinearities does not provide effective protection against black-box recovery attacks.

1 Introduction

Deep Neural Networks (DNNs) have become a cornerstone of modern machine learning, powering applications ranging from image classification to natural language processing. The significant computational resources and proprietary data invested in training these models make their internal parameters highly valuable intellectual property. Consequently, understanding the feasibility and limits of extracting or reverse-engineering DNNs from limited black-box access has emerged as a critical research area with both theoretical and practical implications.

The problem of model extraction has a long history, beginning with the seminal work of Fefferman [8] and later Lowd and Meek [14], who showed that

under certain conditions, the internal structure of simple neural networks can be inferred from input–output observations. More recently, this topic has seen renewed attention, especially for ReLU-based deep neural networks, where black-box adversaries submit chosen inputs to obtain outputs and attempt to reconstruct the architecture and parameters. A notable milestone was reached by Carlini, Jagielski, and Mironov at Crypto 2020 [5], who proposed a differential cryptanalysis-inspired attack requiring only a polynomial number of oracle queries but an exponential amount of time, making it feasible only for networks with at most a few tens of neurons beyond the first hidden layer. The primary bottleneck in their approach was the determination of a single bit of the sign of each neuron’s weight vector, which forced an exhaustive search over an exponentially large space of sign combinations.

Building on this line of work, Canales-Martínez et al. [3] recently demonstrated the first practical polynomial-time attack capable of extracting with high precision the real-valued parameters of deep ReLU networks using only black-box access. Their approach introduces three complementary sign-recovery techniques: a System-of-Equations (SOE) method for early layers, a *Neuron Wiggle* technique for intermediate layers, and a specialized Last Hidden Layer method. Together, these allow the recovery of all neuronal signs without resorting to exponential search. They showcased their attack on a full-scale DNN trained on CIFAR10 with 3072 inputs, eight hidden layers of 256 neurons each (about 1.2 million parameters), replacing the exhaustive 2^{256} sign search per layer required by Carlini et al. with an algorithm completing in approximately 32 minutes on a 256-core machine. This represents the current state of the art in cryptanalytic extraction of ReLU-based deep neural networks.

A parallel line of work has sought to handle the far more restrictive hard-label setting, in which an attacker can only observe the predicted class label rather than full logits. This is the most challenging scenario in the taxonomy introduced by [12]. After several decades without progress beyond tiny models, Chen et al. [7] proposed at Asiacrypt 2024 the first extraction technique in this setting but achieved only exponential-time sign recovery and demonstrated results on networks with at most four neurons. Very recently, Carlini, Chávez-Saab, Hamblitz, Rodríguez-Henríquez, and Shamir [4] introduced new geometric techniques for identifying “dual points” at the intersection of decision and critical boundaries, achieving polynomial-time cryptanalytic extraction of nearly one million parameters from a CIFAR10 ReLU network under the hard-label scenario. Their work shows that even when only class labels are available, the full parameters of large DNNs can be efficiently recovered by analyzing the geometry of decision boundaries.

In contrast to the hard-label setting, our work assumes the attacker still has full access to the DNN’s numeric outputs (logits), and therefore our attacks can exploit richer side-channel information (e.g., derivatives) that remains hidden in the hard-label scenario. This allows us to focus on generalizing and strengthening the attack methodology rather than compensating for information loss.

1.1 Our contributions

Building on and extending prior work, our paper makes the following contributions:

1. We present an attack that recovers both weights and biases in networks with non-linear activation functions that converge to linear functions on both sides, including GELU, SiLU, and Sigmoid, among others. The method is universal in the sense that it does not rely on the specific shape of the activation function, apart from the requirement that the non-linear region be sufficiently narrow for the linear behavior to be observable.
2. As an immediate corollary, our results demonstrate that the original attack of Carlini et al. does not fundamentally depend on the existence of ReLU critical points: the geometric intuition behind adjacent linear regions can be reformulated without appealing to non-differentiability. In practice, this eliminates the *point of non-differentiability* requirement in the original framework and extends its applicability to models where such points are absent.
3. We show that recovering neuron signatures can in some cases be easier with non-linear activation functions than with ReLU. In particular, for several such functions, no special procedure is required to recover the sign of the signature.
4. As a side result, we also demonstrate that the activation functions themselves can be identified when they are not publicly known.

Together, these contributions expand the reach of black-box cryptanalytic extraction and generalize the geometric/differential viewpoint beyond non differentiable activations.

Remark. Our algebraic approach depends on reliably detecting zeros in systems of linear equations. Finite-precision arithmetic and accumulated estimation errors can turn exact zeros into very small nonzero values, so in practice one must apply principled numerical thresholds and, where necessary, re-evaluate borderline cases. Thus, while algebraically cleaner, the method still requires engineering care for robust, large-scale deployments.

2 Definitions and Related Works

2.1 Notation and definition

In this section we formalize the definition used throughout the paper, following the notations introduced in previous works.

Definition 1 (Deep Neural Network). *An r -layer deep neural network $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ with parameters θ is a composition of alternating linear maps and non-linear activations:*

$$f_\theta = f_{r+1} \circ \sigma \circ f_r \circ \sigma \cdots \circ \sigma \circ f_1,$$

where each f_i is an affine transformation and σ acts componentwise.

Definition 2 (Layer). The i -th fully connected layer $f_i : \mathbb{R}^{d_{i-1}} \rightarrow \mathbb{R}^{d_i}$ is defined by

$$f_i(x) = A^{(i)}x + b^{(i)},$$

where $A^{(i)} \in \mathbb{R}^{d_i \times d_{i-1}}$ is the weight matrix and $b^{(i)} \in \mathbb{R}^{d_i}$ is the bias vector.

Definition 3 (Neuron). The j -th neuron in layer i computes

$$\eta_{i,j}(x) = \sigma(A_j^{(i)}x + b_j^{(i)}),$$

where $A_j^{(i)}$ is the j -th row of $A^{(i)}$ and $b_j^{(i)}$ is the j -th entry of $b^{(i)}$.

Definition 4 (Architecture). The architecture of a fully connected DNN is given by the input dimension d_0 , the number of layers r , and the width d_i of each hidden layer i and of the output layer d_{r+1} .

Definition 5 (Parameters). The parameters of the DNN are $\theta = \{A^{(i)}, b^{(i)} : i = 1, \dots, r+1\}$.

Definition 6 (Neuron State). For a neuron η and input x , let $V(\eta; x)$ denote the pre-activation value before σ . We say η is **active** at x if $V(\eta; x) > 0$ and **inactive** otherwise. The state is denoted $S(\eta; x)$.

Definition 7 (Critical Point). An input x is a critical point for neuron η if $V(\eta; x) = 0$.

Definition 8 (Linear Region). The linear region of an input x is the set

$$\{u \in \mathcal{X} \mid S(\eta; u) = S(\eta; x) \text{ for all neurons } \eta\}.$$

Inside a linear region the DNN behaves as an affine map.

2.2 Hidden-Layer Extraction in Deep Neural Networks

A central component in the cryptanalysis of deep neural networks (DNNs) is the extraction of hidden layers from black-box access. Conceptually, recovering a hidden layer means reconstructing, for every neuron in the layer, both its weight vector and bias term. This task is typically divided into two subproblems:

1. *Signature recovery*, i.e., determining each neuron's weight vector up to an unknown scaling factor (often referred to as the neuron's *signature*).
2. *Sign recovery*, i.e., resolving the orientation ambiguity (the $\pm$ factor) so as to uniquely determine the neuron's hyperplane and its action on inputs.

Together, these steps yield the exact affine map implemented by the hidden layer, thereby allowing the adversary to *peel off* the layer and go deeper into the network.

Signature Recovery. The geometric foundation of signature recovery exploits the fact that piecewise-linear activations such as ReLU partition the input space

into linear regions, separated by *critical hyperplanes* where a neuron switches state. Inside each region the DNN behaves as an affine function. By identifying points near the critical hyperplanes in which neurons switch states, and comparing affine responses across regions, one can construct linear equations whose unknowns are the neuron parameters. Solving these systems yields the neuron weight vectors up to scalar multiples. Carlini, Jagielski, and Mironov [5] first demonstrated that signature recovery can be performed with a polynomial number of oracle queries.

In the more restrictive *hard-label* setting, where only the predicted class label is observable, the detection of critical points is impossible in the usual sense. Recent work introduced the notion of *dual points* which are inputs that simultaneously lie on a neuron’s critical hyperplane and on the decision boundary of the network. By carefully probing decision boundaries to identify dual points, one can still recover signatures, though at the cost of more elaborate geometry and searching.

Sign Recovery. Once signatures are known, the main challenge is to determine the correct orientation of each weight vector. This problem was long regarded as the bottleneck to practical extraction, as the naive approach requires an exponential search over sign combinations. Several families of techniques have been proposed to address this:

Statistical perturbation methods. The *Neuron Wiggle* technique of [3] perturbs inputs locally in directions aligned with a candidate weight vector. By observing the differential change in network outputs across the two sides of a hyperplane, one can statistically infer the neuron’s orientation. Aggregating votes over many such perturbations yields the sign with high probability. This approach is efficient in practice but depends on tunable thresholds, sample sizes, and may fail for neurons whose weight vectors are closely aligned with others.

Algebraic methods. Alternative strategies leverage systems of linear equations whose solutions implicitly determine neuron signs. For early layers, linear dependencies among activation patterns can be exploited (the so-called SOE method), while for the last hidden layer, second-order derivatives yield direct algebraic conditions. These methods are more “clean” theoretically, as they avoid statistical thresholds, but they require careful numerical handling: due to finite precision, exact zeros are often replaced by small nonzero residues.

Geometric approaches in the hard-label setting. When only class labels are observable, sign recovery cannot rely on logits or derivatives. Instead, adversaries analyze the geometry of decision boundaries near dual points. By combining local consistency checks across many such points, the correct orientation of neurons can be inferred in polynomial time [4]. This represents the first polynomial-time hard-label sign recovery on networks of nontrivial size.

Practical fallback strategies. Despite these advances, some neurons remain “hard” to classify with high confidence, especially when weight vectors are clustered.

Practical implementations therefore deploy fallback strategies: low-confidence neurons are either brute-forced in small groups, with correct assignments identified in subsequent layers, or are resolved by parallelizing extraction under competing sign hypotheses and discarding inconsistent branches early. Empirical evaluations show that the number of such hard neurons per layer is typically small, making these strategies scalable in practice [9].

3 Signature Recovery with Non-linear Activation Functions

The Rectified Linear Unit (ReLU) has become the de facto standard activation function in modern deep learning due to its simplicity, computational efficiency, and ability to mitigate the vanishing gradient problem. Nevertheless, its piecewise-linear nature and non-differentiability at zero raise both theoretical and practical concerns, motivating the study of alternative nonlinearities.

Several variants of ReLU introduce smoothness or modify its behavior near the origin. The *Leaky ReLU* allows a small slope for negative inputs instead of hard zeroing, thereby preserving gradient flow for negative inputs. The *Parametric ReLU (PReLU)* generalizes this idea by learning the slope parameter during training. The *Exponential Linear Unit (ELU)* and its smoother variant, the *Scaled ELU (SELU)*, replace the hard kink with an exponential curve for negative inputs, which encourages mean activations closer to zero and can improve convergence. The *Swish* and *Mish* functions introduce smooth, non-monotonic alternatives that have shown empirical gains in vision tasks. Overall, the exploration of ReLU alternatives reflects an ongoing effort to balance mathematical tractability, numerical stability, and empirical performance.

3.1 Recovering the critical hyperplanes

In this section, we propose a new technique to recover the signature of a neuron associated with a non-linear activation function. For the ReLU activation function, signature recovery always begins with the detection of critical points. Given two sufficiently distant points, one can determine the linear maps induced by the DNN around each of them. If the maps differ, then at least one critical point lies between the two points, and all such points can be identified via dichotomy.

However, when the activation function is not piecewise linear, the above technique fails. In this work, we assume that the activation function σ converges exponentially fast to a linear (or constant) function in both directions and is non-linear only in a neighborhood of 0. In this case, the state of a neuron is no longer strictly active or inactive, and the linear regions of the DNN are separated by non-linear zones rather than by critical hyperplanes.

Informally, non-linearity can be interpreted as a fuzzy expansion of the neuron’s linearity. Around each point, every neuron computes a linear combination of its inputs based on the outputs of the preceding neurons. Geometrically, this

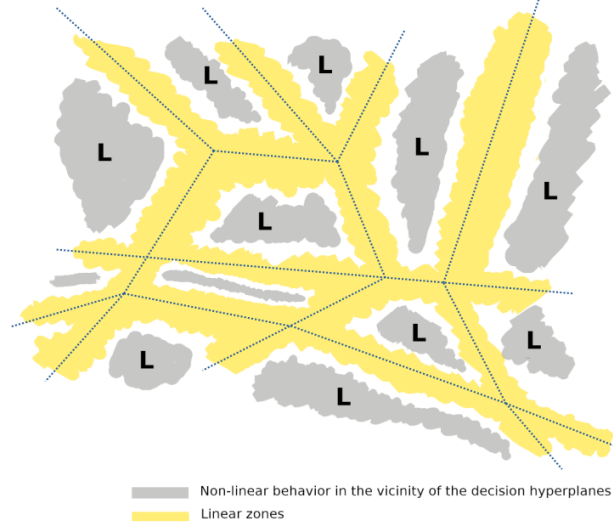


Fig. 1: A 2D view of the geometry of a DNN with non-linear activation functions.

linear combination can be locally represented by a hyperplane, and each neuron radiates a non-linear zone around it as sketched in Figure 1. Due to the training process, these hyperplanes are arranged in a way that optimizes the shapes of the resulting cells. While the non-linear zones tend to overlap, their influence gradually diminishes with distance from the hyperplane, eventually decaying into pure linearity, particularly near the centers of the cells.

The linear behavior of DNNs is fundamental: as a consequence of training, it is virtually guaranteed that whenever two hyperplanes intersect, their respective non-linear zones can never completely overlap.

To explain our new method for recovering the signature of a neuron η_j in the first layer, we begin with a simple observation. Let x^* be a point such that all neurons except η_j behave linearly. (This setting can be seen as a relaxed version of the notion of critical points from [5]). Near such a point, the DNN can be expressed as

$$F(x) = \alpha \sigma(A_j^{(1)}x + b_j^{(1)}) + L(x) + \beta, \quad (1)$$

where α and β are unknown scalar constants, and L is an unknown linear map.

To understand Equation (1), first recall that we are assuming the DNN output has dimension only one. This is without loss of generality, since a higher-dimensional output yields strictly more information. Also note that since only one neuron behaves non-linearly, its input is some affine map $A_j^{(1)}x + b_j^{(1)}$ of the DNN input x ; hence its output is of the form $\sigma(A_j^{(1)}x + b_j^{(1)})$. Its contribution

to the final DNN output must be an affine map of this output, hence it is of the form $\alpha \sigma(A_j^{(1)} x + b_j^{(1)}) + \beta_0$ for some scalars α and β_0 . Finally, other neurons from the same layer behave linearly, which implies that their contribution to the final DNN output can be expressed as an affine map $L(x) + \beta_1$. Setting $\beta = \beta_0 + \beta_1$ yields the equation.

The key fact is that the second derivatives of F depend only on the activation function and the neuron signature. In particular,

$$\frac{\partial^2 F}{\partial x_i \partial x_k}(x) = \alpha A_{j,i}^{(1)} A_{j,k}^{(1)} \sigma''(A_j^{(1)} x + b_j^{(1)}). \quad (2)$$

Since, by definition, σ is non-linear in a neighborhood of x^* , we have $\sigma''(x^*) \neq 0$. This allows us to compute the normalized row of weights using the relation

$$\frac{\frac{\partial^2 F}{\partial x_i \partial x_k}(x^*)}{\frac{\partial^2 F}{\partial x_k \partial x_\ell}(x^*)} = \frac{A_{j,i}^{(1)}}{A_{j,\ell}^{(1)}}. \quad (3)$$

Hence, compared to the ReLU case, the non-linearity of the activation function enables us to recover the neuron signature without first locating an exact critical point. In practice, it suffices to find a point x^* that is sufficiently close to a given critical hyperplane, so that its influence on x^* dominates that of the other hyperplanes associated with the remaining neurons. Since typical non-linear activation functions converge exponentially fast to linear or constant functions, such points constitute a non-negligible proportion of the input space.

A very simple sieve can thus be implemented: to test whether an input point of the DNN is in the favorable situation close to a critical hyperplane discussed earlier, we compute Equation 3 with different values of k , and check whether the results are (nearly) identical. If so, then the point is assumed to be close to a critical hyperplane (later steps of the algorithm will be able to discard false positives). On average, such points can be found efficiently, with a success rate varying between 25% and 40% depending on the norm and the exact method of computation (a detailed discussion is provided in B). (In principle, it would also be possible to use the dichotomy process introduced by Carlini et al., except instead of comparing the linear maps around two points, we would compare the critical hyperplanes that influence them.)

The algorithm was tested successfully against GELU, SiLU, and even non-rectifier functions such as Sigmoid and tanh, which are outside the original scope of the attack. As long as the activation function is non-linear, this technique applies, which constitutes one of the main contributions of this work. Further implementation details are provided in Section 4.

Efficiency. Since the distribution of parameters in a trained neural network does not follow any simple formula, any complexity analysis must rely on simplifying modeling assumptions. In the spirit of Carlini et al., we adopt the assumption that when sampling points in the input space of the network, the probability to activate each neuron is roughly uniform. We also assume that for each sample, obtaining a usable output (as opposed to hitting a region of space where the

non-linear influence zones of several neurons interfere, for example) occurs with probability lower-bounded by a constant. This is supported by experiments, as long as the sampling process is performed carefully enough: see Section 4 for more details. Under those assumptions, detecting every neuron amounts to a coupon collection problem over n neurons, which implies that we need to sample $O(n \log n)$ points to collect the signature of every neuron. The processing time for each sample is clearly polynomial. This model is quite crude, and empirical validation is the main argument. We mainly point this out to emphasize that there is no hidden exponential complexity.

3.2 Improving the quality of signatures

The quality of the signature obtained through the previous technique depends on how close the point x^* lies to the critical hyperplane of the target neuron and how far it is from other critical hyperplanes. Therefore, the recovered signature can be used to identify a new point closer to the target hyperplane, since the nearer the point is to this hyperplane, the weaker the non-linear contributions from the other hyperplanes become.

Although the bias vector $b^{(1)}$ is still unknown, the knowledge of a noisy $A_j^{(1)}$ immediately provides a pseudo-normal vector N_j that can be used to construct new candidate points of the form $x^* + \epsilon N_j$.

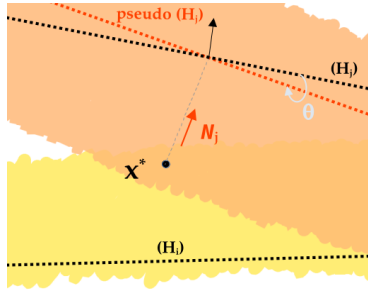


Fig. 2: Denoising $A_j^{(1)}$ by following the pseudo-normal.

By definition, the pseudo-normal is itself a source of error, since in each plane the translation vector will deviate by an angle θ , as illustrated in Figure 2. When the noise is strong, some coordinates of N_j may even be missing. Nevertheless, it still provides a useful approximation of the true hyperplane H_j . The technique can be refined in the following ways:

1. As the estimate of $A_j^{(1)}$ improves, we update N_j accordingly, which accelerates convergence toward the true hyperplane H_j .
2. Lateral moves may also be attempted. Indeed, approaching the hyperplane may inadvertently bring us closer to an intersection with another hyperplane.

To avoid this, it is natural to also consider candidate points of the form $x^* + \epsilon v$ for $v \in H_j$.

This method performs well when the network operates with multi-precision arithmetic. However, under limited-precision settings, the computation of second derivatives introduces additional noise that this technique cannot eliminate.

3.3 Adjacent linear zones

For more realistic targets such as TensorFlow, an alternate technique is required. From a point x^* in a non-linear zone, we follow the pseudo-normal N_j in both directions in order to reach the linear zones adjacent to the non-linear one as depicted in Figure 3. Detecting whether we have reached a linear zone is done simply by checking that the gradient of the DNN output around the current point is (close to) constant.

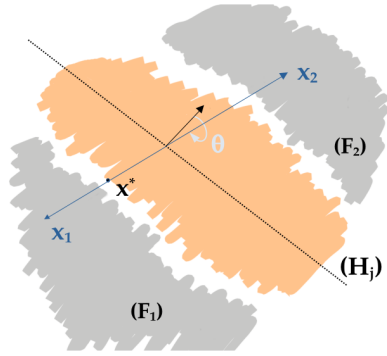


Fig. 3: Denoising by exploiting the adjacent linear zones

The main point of this technique is that so-called linear zones typically have very low noise arising from non-linear activation functions. This is because typical activation functions converge exponentially fast towards affine maps at both $-\infty$ and $+\infty$ (see the next section). As a result, we observe experimentally that the affine maps approximating the DNN output locally around the target point in a linear zone have very low noise (in the sense that the affine maps recovered experimentally are extremely close to their correct values). This helps extract neuron weights with high precision.

The exact way we exploit these *adjacent linear zones* depends on the activation function (cf. next section), but relies on the behavior of F in the whole area given by Eq. 1:

$$F(x) = \alpha \sigma(A_j^{(1)} x + b_j^{(1)}) + L(x) + \beta.$$

3.4 Rectifiers

Let us assume that $\sigma(x)$ converges to the function $x \mapsto ax$ when $x > 0$ and to $x \mapsto c$ when $x < 0$. In this case, the linear maps in the two regions are

$$\begin{aligned} - F_1 : x &\mapsto \alpha a A_j^{(1)} x + \alpha a b_j^{(1)} + L(x) + \beta, \\ - F_2 : x &\mapsto \alpha c + L(x) + \beta. \end{aligned}$$

If $c = 0$ (as in the case of the GELU and SiLU activation functions), then $F_1 - F_2$ directly yields the normalized signature of η_j , including the bias $b_j^{(1)}$. Since both F_1 and F_2 can be easily recovered by querying the DNN, this provides a significantly more accurate estimate of the neuron signature. Although the precision is not absolute, it is an order of magnitude higher than any calculation based on second-order derivatives.

More generally, even when $c \neq 0$ (as with the SELU activation function), $F_1 - F_2$ still reveals a more precise normalized value of $A_j^{(1)}$. Let x^* be a solution to the affine equation $F_1(x) - F_2(x) = 0$. Then we obtain

$$b_j^{(1)} = \frac{c}{a} - A_j^{(1)} x^*.$$

Hence, in both cases, we have access to the function

$$\tilde{F}(x) = F(x + x^*) = \alpha \sigma(A_j^{(1)} x + c/a) + L(x) + \tilde{\beta},$$

where α , L , and $\tilde{\beta}$ are unknown, while $A_j^{(1)}$ is known up to a scalar multiple. (Note that in this expression, the constant arising from $L(x^*)$ has been folded into $\tilde{\beta}$.)

In the case of a piecewise-linear activation function, we have $\alpha \sigma(x) = \sigma(\alpha x)$ whenever x and αx lie in the same linear region. Consequently, to reconstruct an equivalent DNN it suffices to determine in which region $A_j^{(1)}$ resides. For example, with ReLU it is enough to recover the sign of a single component of $A_j^{(1)}$. By contrast, for smooth non-linear activation functions it is necessary to recover the exact value of at least one component of $A_j^{(1)}$; otherwise, the network's output in the non-linear zones will differ from that of the original model.

A general approach is to analyze the derivatives of $\tilde{F}$. The locations of the local extrema of the first derivative are independent of all unknowns in the expression of $\tilde{F}$, and each extremum (provided it is not located at c/a) reveals a unique value of $A_j^{(1)}$. Moreover, because σ'' vanishes outside the non-linear region and σ is genuinely non-linear, $|\sigma''|$ must attain local maxima within that region. In the worst case this maximum occurs at c/a , but then the third derivative necessarily exhibits a local extremum elsewhere (or is constant, which would imply that the non-linear part of σ is in fact quadratic, in which case the breakpoints in the second derivative can be exploited to recover $A_j^{(1)}$).

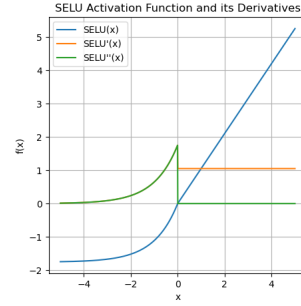
Therefore, for all activation functions of the considered form, one can always determine the possible values of $A_j^{(1)}$ by examining the local extrema of the first, second, or third derivatives. In the following we review several common

non-linear activation functions and explain, for each, how and to what extent $A_j^{(1)}$ can be recovered.

SELU. The Scaled Exponential Linear Unit activation function is defined as

$$\sigma(x) = \max(0, x) + \min(0, \alpha(e^x - 1)).$$

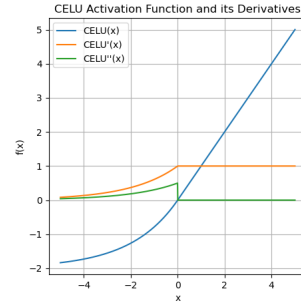
Its first derivative is non-linear and strictly positive for negative values of x , while it is constant and equal to 1 for $x > 0$. Consequently, identifying the break point directly reveals both the value of $A_j^{(1)}$ and its sign. (Note: the first and second derivatives of SELU are equal on negative values, so that they overlap on the diagram to the right.)



CELU. The Continuously Differentiable Exponential Linear Unit (CELU) activation function is defined as

$$\sigma(x) = \max(0, x) + \min(0, \alpha(e^{x/\alpha} - 1)).$$

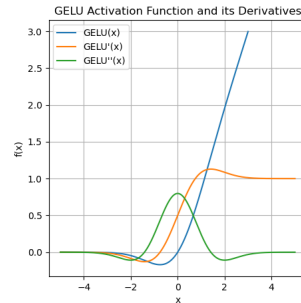
It is similar to the SELU activation function, but its first derivative is continuous. However, the discontinuity of its second derivative at $x = 0$ can be exploited to recover both the value of $A_j^{(1)}$ and its sign.



GELU. The Gaussian Error Linear Unit activation function is defined as

$$\sigma(x) = \frac{x}{2} \left(1 + \operatorname{erf} \left(\frac{x}{\sqrt{2}} \right) \right).$$

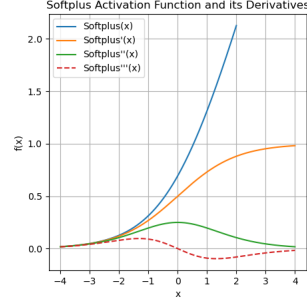
Its second derivative attains its maximum at $x = 0$, which therefore cannot be used to deduce the exact value of $A_j^{(1)}$. There are two possible approaches to recover $A_j^{(1)}$, but due to the symmetry of the second derivative, the sign cannot be determined by these methods. The most accurate approach is to locate the extrema of its first derivative, which occurs at $x = \pm\sqrt{2}$.



Softplus. This function is a smooth approximation of the ReLU activation and is defined as

$$\sigma(x) = \frac{1}{\beta} \log(1 + e^{\beta x}).$$

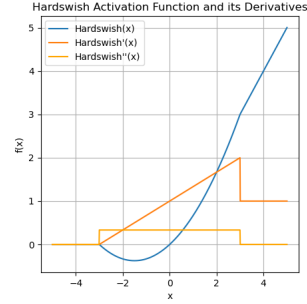
To recover $A_j^{(1)}$, one must compute its third derivative and identify one of the two local extrema. However, as in the case of the GELU function, it is not possible to determine the exact sign of $A_j^{(1)}$, since the second derivative is an even function.



Hardswish. The Hardswish activation function is defined as

$$\sigma(x) = x \frac{\text{ReLU6}(x + 3)}{6},$$

where ReLU6 is a clipped version of the ReLU function given by $\text{ReLU6}(x) = \min(\max(0, x), 6)$. As in the case of SELU, the discontinuity of its first derivative at $x = 3$ can be exploited to recover both the exact value of $A_j^{(1)}$ and its sign.



3.5 Non-Rectifiers

Let us now assume that $\sigma(x)$ converges to the constant function $x \mapsto c_1$ when $x > 0$ and to $x \mapsto c_2$ when $x < 0$, as is the case for the Sigmoid activation function. In this setting, we have

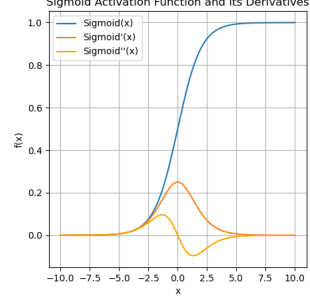
$$F_1 - F_2 = \pm \alpha (c_1 - c_2),$$

which reveals the coefficient α up to its sign. In addition, this case also provides the linear map L (which will be constant equal to zero if all activation functions are non-rectifiers). Using this information, the quality of the estimate of $A_j^{(1)}$ can be further improved by examining the first-order derivatives of F .

Sigmoid. This activation function is one of the most well-known and is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Since $\sigma(-x) = 1 - \sigma(x)$, there is no need to recover the sign of $A_j^{(1)}$. The location of the maximum of the first derivative of F uniquely determines $b_j^{(1)}$, while the value of this maximum yields the exact value of $A_j^{(1)}$. Several related activation functions appear in the literature, such as tanh and Softsign, and the same procedure can be applied to fully determine the corresponding neuron parameters.



3.6 Sign Recovery

The techniques presented in Section 3.4 fail to recover the sign for two activation functions: GELU and Softplus. This is due to an inherent impossibility when using the type of algebraic approach developed in the previous section, arising from a specific form of symmetry. We now define and explain the problem. In the next section, we will discuss how it can be circumvented.

Recall that when attempting to recover the signature of a neuron, we cannot directly query its output. Instead, what we observe is the output of the target neuron combined with the influence of other neurons in the network. Recall that during the attack, queries are arranged so that the other neurons behave linearly in the chosen region of the input space. In this situation, the observed output takes the form

$$g = a \circ \sigma \circ b + c,$$

where $a : \mathbb{R} \rightarrow \mathbb{R}$, $b : \mathbb{R}^n \rightarrow \mathbb{R}$, and $c : \mathbb{R}^n \rightarrow \mathbb{R}$ are affine maps.

Definition 9. An activation function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is said to be affine-symmetric if there exists an affine map $\lambda : \mathbb{R} \rightarrow \mathbb{R}$ such that for all $x \in \mathbb{R}$,

$$\sigma(-x) = \sigma(x) + \lambda(x).$$

Remark. If a and b are invertible affine maps $\mathbb{R} \rightarrow \mathbb{R}$, then σ is affine-symmetric if and only if $a \circ \sigma \circ b$ is affine-symmetric. In other words, affine-symmetry is preserved under affine equivalence: composing σ with affine maps on the input or output does not affect whether it is affine-symmetric.

Now suppose that we have identified affine maps a , b , and c such that $g = a \circ \sigma \circ b + c$. Writing $a(x) = a_1x + a_0$, we observe that if σ is affine-symmetric,

with $\sigma(-x) = \sigma(x) + \lambda(x)$, then for all x we have:

$$\begin{aligned}
g(x) &= a\left(\sigma(b(x))\right) + c(x) \\
&= a\left(\sigma(-b(x)) - \lambda(b(x))\right) + c(x) \\
&= a_1\sigma(-b(x)) + a_0 - a_1\lambda(b(x)) + c(x) \\
&= a\left(\sigma(-b(x))\right) + d(x),
\end{aligned}$$

where $d(x) = c(x) - a_1\lambda(b(x))$ is affine.

Thus, if σ is affine-symmetric, any map of the form $g = a \circ \sigma \circ b + c$ can also be written as $g = a \circ \sigma \circ (-b) + d$. It follows that using purely algebraic means, and without prior knowledge of the affine map c , it is impossible to recover the sign of b : the two forms are indistinguishable.

The ReLU function is affine-symmetric, which explains why sign recovery was a challenging problem in [5]. The neuron-wiggling technique introduced in [3] addresses this issue by using a *statistical* approach rather than a purely algebraic one. The key idea is to treat c as an independent random variable, e.g. noise, so that the inherent asymmetry of ReLU (its larger slope on positive inputs than on negative inputs) can still be detected *on average*.

The case of non-linear maps When the activation function σ is not ReLU, one might hope that it is easier to recover the sign of b , by exploiting the fact that the graphs of $x \mapsto \sigma(x)$ and $x \mapsto \sigma(-x)$ generally have different “shapes” (up to affine shifts). Surprisingly, this intuition fails for several important non-linear activation functions. Overall, non-linear activation functions can be split into three cases.

1. **Affine-symmetric functions.** GELU and Softplus (as well as SiLU) are affine-symmetric. This is apparent from the fact that their second derivative is an even function:

$$\begin{aligned}
&\forall x, \sigma''(x) = \sigma''(-x) \\
&\Leftrightarrow \forall x, \sigma''(x) - \sigma''(-x) = 0 \\
&\Leftrightarrow \exists \lambda_0, \lambda_1 \in \mathbb{R}, \forall x, \sigma(x) - \sigma(-x) = \lambda_1 x + \lambda_0 \\
&\Leftrightarrow \sigma \text{ is affine-symmetric.}
\end{aligned}$$

Remark. This chain of equivalences assumes that σ'' is well-defined everywhere, which holds for GELU, Softplus and SiLU.

Concretely, this means that sign recovery for GELU and SiLU encounters the same fundamental obstacle as ReLU, discussed earlier for ReLU: purely algebraic approaches exploiting a single region cannot break the symmetry between solutions with opposite signs. However, the statistical neuron-wiggling technique again applies: away from the origin, their derivatives are

close to 1 for positive inputs and close to 0 for negative inputs, which provides a statistical signal for distinguishing the sign.

2. **Odd functions (up to a constant).** The sigmoid and arctan activation functions are odd functions, up to the addition of a constant: $\sigma(x) = C - \sigma(-x)$ for some constant C . This means that the sign of the neuron can be chosen arbitrarily; the neural network carries no information about that sign. This is the case discussed in Section 3.5.
3. **Asymmetric functions.** For all other activation functions we have encountered, it is possible to recover the sign using the algebraic approach from Section 3.4, by exploiting the asymmetry of the behavior of the function between its positive and negative inputs. See the discussion for each function in that section.

An algebraic approach The analysis so far shows that activation functions are split into three cases: those where the sign is undefined; those where it is defined and can be recovered using local algebraic techniques from Section 3.4; and those where it is defined but cannot be recovered using such techniques. This last case is limited to affine-symmetric functions such as GELU and Softplus. We now explain a different approach for recovering the sign in those cases, with the caveat that it is only applicable when the neural network is expanding (defined next). The technique also exploits the fact that the only affine-symmetric functions we have encountered are rectifiers: they tend to a constant over negative inputs, and towards identity over positive inputs.

Let m denote the dimension of the vector space at the input of the targeted layer, which is also equal to the number of neurons of the layer. Let n denote the dimension of the vector space at the output of the layer. If $m \leq n$, we say that the target layer is *expanding* (in the loose sense: $m = n$ is included).

For simplicity, focus on the case $m = n$, when targeting the first layer; the general case is similar. As before, we are interested in a target neuron, and using techniques from the previous section, we have identified the hyperplane associated with that neuron (that is, the hyperplane of equation $f(x) = 0$, where f is the affine functional at the input of the neuron). Determining the sign of the neuron means determining which of the two half-spaces on each side of the hyperplane corresponds to positive inputs for the activation function of the neuron (while the other half-plane corresponds to negative inputs).

For $1 \leq i \leq n$, let $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ denote the affine functional at the input of each neuron in the target layer. Let $\omega_i : \mathbb{R}^n \rightarrow \mathbb{R}$ denote the linear component of f_i , that is, ω_i is a linear functional such that $f_i(x) = \omega_i(x) + c_i$ for some constant $c_i \in \mathbb{R}$. Assume without loss of generality that the neuron we are targeting corresponds to $i = 1$. There are $n - 1$ functionals ω_i for $i > 1$. Hence there must exist a non-zero vector v that is mapped to zero by all functionals ω_i for $i > 1$. This vector v has the following special property: for any point x the half-space where the target neuron $i = 1$ is inactive, $\omega_i(x + v) = \omega_i(x)$ for all i , hence $f_i(x + v) = f_i(x)$ for all i . This immediately implies that in that half-space, $F(x + \lambda v) = F(x)$ for all $\lambda \in \mathbb{R}$ such that $x + \lambda v$ remains in the same

half-space. (Here, F denotes as usual the output of the entire network.) This holds over the entire half-space.

On the other half-space, there is no reason for such a vector v to exist. In fact, if it existed, it would mean that one dimension in the input of the network is systematically ignored, which seems unlikely. This leads to a distinguisher: In order to determine over which half-space the target neuron $i = 1$ is active, find for which of the two half-spaces there exists a non-zero vector v such that $F(x + \lambda v) = F(x)$ for all x and $x + \lambda v$ in the half-space.

This reduces the problem to finding such a vector v whenever it exists. To do so, the most natural algorithm would be to find many regions of the targeted half-space where F behaves as an affine map. The vector v must lie in the kernel of the linear part of every one of those affine maps. Hence, computing the kernel of each map and taking the intersection yields (the span of) v . On the other half-space, where the neuron $i = 1$ is active, such an intersection is empty, as long as we have sample enough regions. (More precisely, we need to have found enough regions such that every neuron is active in at least one region.)

This algorithm gives the general idea, but does not work in an actual implementation because it is not numerically stable: small errors in computation due to floating-point arithmetic result in the intersection of kernels being empty on both half-spaces. This is avoided with a slightly more complicated algorithm: instead of taking the intersection of the kernels, we stack vertically in a matrix vectors $(F(x + e_1) - F(x), \dots, F(x + e_n) - F(x))$, where x is sampled from the target half-space, and e_i is the canonical basis. In theory (if computations over the reals were perfect), v would lie in the kernel of that matrix. This implies in particular that the matrix has rank at most $n - 1$, hence it must have zero as a singular value. In practice (with approximation errors due to floating-point arithmetic), what happens is that one of the singular values of the matrix is extremely small. The neuron is considered inactive whenever a singular value below the appropriate threshold is found. Singular values can be computed efficiently using a Singular Value Decomposition (SVD) algorithm. This method has a very high success rate in practical (see Table 1).

Table 1: Success rate of our SVD based 1st layer distinguisher primitive. This technique is currently only implemented on the high precision backend. As such the tests are limited to small networks where the computation is not too expensive.

Target Network	GELU	SELU	SiLU	Mish
$4 \times 4^4 \times 4$	>99%	>99%	>99%	>99%
$6 \times 5^3 \times 4$	>99%	>99%	>99%	>99%

3.7 Identification of the activation function

Throughout this article, we assume that the activation function of each layer is known in advance (e.g., ReLU, GELU, sigmoid, or another standard activation). However, the attack naturally extends to the case where the activation functions are not known beforehand. This relies on two key facts:

1. For each layer, the first step of our attack is to identify (an approximation of) the hyperplanes corresponding to each neuron. This step does not depend on the activation function.
2. Once the hyperplane corresponding to a neuron is known, it is possible to identify the activation function of that neuron.

The first fact follows directly from the description of the attack. The second fact requires justification, but let us first assume it holds. In that case, the attack can be adapted to determine the activation function of each neuron after recovering its hyperplane. This shows that concealing activation functions does not strengthen security against our attack.

Recovering the activation function. We now explain why the second fact is true. For simplicity, consider a neuron on the first layer, the argument extends straightforwardly to deeper layers. Let σ denote the unknown activation function of the neuron. Knowing the signature of its hyperplane means that we know a linear map f such that the neuron outputs $\sigma \circ f$. More precisely, because we only recover the *signature*, we do not know f exactly, but only up to composition with an affine map on its output. That is, we know $g = \phi \circ f$, where $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is an unknown affine map.

Moreover, we cannot observe $\sigma \circ f$ directly. Since our attack isolates a region of the input space where subsequent layers behave almost affinely, the observable function is of the form $\psi \circ \sigma \circ f$, where ψ is another unknown affine map arising from the action of deeper layers.

In summary, we can only query

$$\psi \circ \sigma \circ \phi^{-1} \circ g,$$

and since g is known, this reduces to query access to $\psi \circ \sigma \circ \phi^{-1}$.

Recognition of the activation function. Although we cannot query σ directly, the problem of identifying it from $\psi \circ \sigma \circ \phi^{-1}$ is straightforward: plotting the curve of $\psi \circ \sigma \circ \phi^{-1}$ reveals the shape of σ , up to affine translation and scaling. One can then compare it against a list of standard activation functions. This recognition can also be automated by interpolating and selecting the best match between the queried curve and the standard functions.

It is worth noting that $\psi \circ \sigma \circ \phi^{-1}$ is *affine-equivalent* to σ : the two differ only by affine transformations of their input and output. The problem of recognizing affine equivalence, and finding the affine maps that witness it, has been studied in the context of white-box cryptography [2,15]. This further reinforces the connection between white-box cryptanalysis and the cryptanalysis of neural networks, already emphasized in [4].

3.8 Recovering deeper layers

So far, we have presented techniques to recover neurons and their associated weights for the first neural layer. The attack on the first layer can be extended to deeper layers using the same “peeling-off” approach as [5]. This works essentially in the same way as prior work, and we only briefly sketch the process here.

As the name suggests, the peeling-off technique consists in canceling the contribution of the first n layers, in order to target the $(n + 1)$ -th layer. For any given point at the input of the DNN, canceling the contribution of the first n layers is possible, since we have already recovered the weights of the first n layers. At a high level, once the influence of the first n layers is compensated in this way, everything should behave as if we were attacking the first layer. This means we can use the same technique we have already developed for the first layer.

In reality, this process is not as straightforward as it seems. We typically want to target points in critical zones close to the hyperplanes of a target neuron from the $n + 1$ -th layer, and far from the hyperplanes of other neurons. This ensures that only the target neuron contributes non-linearly in the vicinity of the chosen point. Two obstacles arise:

1. First, inactive neurons from earlier layers decrease the effective dimension of the input space around the chosen point. As a toy example, suppose that we have four neurons on the first layer. If two of them are inactive around the chosen point in the critical zone, then our effective input dimension around that point is only two. This implies that we can only hope to recover at most two weights of the targeted neuron. This is called a partial signature in the work of Carlini et al.
2. Second, we need to be able to identify and recombine partial signatures belonging to the same neuron, in order to eventually fully reconstruct the neuron.

Both issues can be addressed using the same techniques as in [5], and we do not detail it further here. In order to validate that the same techniques apply, we have run experiments on small networks featuring non-linear activation functions, using our technique for dealing with general activation functions, combined with the peeling-off technique from earlier work. These experiments have successfully recovered weights up to the fourth layer. Detailed results can be found in Section 4.3.

4 Implementation details of the attack

In this section we describe our experimental setup and results. Following the methodology of [5], all attacks were implemented in a Python framework and support two backends. The first backend represents an idealized environment where numerical precision is not a limitation; it relies on the `mpmath` library [16] configured to 30 digits of precision. The second backend uses TensorFlow [1] and

therefore operates in double-precision floating point (`float64`). In particular, we discuss how the attacks were adapted for the latter setting, and specifically:

1. where 64-bit arithmetic introduces new constraints;
2. an additional primitive to distinguish signatures of the layer under attack within the pool of recovered signatures;
3. efficient procedures for selecting the correct candidates from a noisy pool of signature candidates.

All attacks were run on a five-year-old Intel Core i7 laptop using a single core. Because some target networks are large, several routines (notably the sieving procedure) were optimized with Cython. In particular in the corresponding experiments (4.3), all tests conducted using Cython are identified by an asterisk (*) placed directly in front of the network class. The implementation is available in the following anonymous repository for now and will be made available under an open-source license when this article is published:

https://github.com/mstealercryptocrypto-ops/mod_stealer26

4.1 Precision issues and calibration process

In [5] the authors studied precision issues in the ReLU setting in depth, focusing primarily on first-order derivatives and linear-algebraic techniques. Although they mention a second-order quantity at critical points, that object is not a true second-order derivative in the sense we use here. By contrast, our attacks rely heavily on second-order derivatives (and, in places, higher-order derivatives, see Appendix A), in order to handle more general non-linear activation functions. Viewed this way, our techniques complement the approaches of [5,3]: we exploit similar geometric and algebraic ideas to recover final signatures, but we push them into regimes that require higher-order differential information. Note also that the level of precision required in the initial non-linear stage of the attack is lower than in later denoising stages, where the precision of recovered signatures is refined subsequently.

In our implementation we compute derivatives using Taylor-series finite-difference formulas following Fornberg’s construction [10]. This requires choosing the stencil size (number of points) and the finite-difference steps ϵ_i used in each formula. Empirically we observed no clear advantage to using distinct values for these three steps, or to adapting them dynamically; using a single carefully chosen ϵ is usually sufficient. The choice of ϵ must be made with care, as emphasized in [5]. Richardson extrapolation [17] can improve accuracy when combined with Taylor formulas, at the cost of additional function evaluations. Further discussion and recommended formula/step choices appear in Appendix B.

Finally, the norm $\|x_i\|$ of the probing vectors is an independent and equally important tuning parameter because it strongly affects the probability of observing a usable signature during sieving. Our experiments indicate the existence of an optimal norm that increases with the input dimension of the layer; choosing

a norm far from this optimum can cause the sieve to fail. This behavior and practical tuning heuristics are discussed in Appendix B. Since both ϵ and $\|x_i\|$ are easy for an adversary to determine empirically, we recommend a short bootstrap phase to calibrate these parameters before launching the main extraction routine.

4.2 Distinguishing the neuron signatures efficiently

An important implementation problem is deciding whether a recovered signature truly belongs to the targeted layer, a detail that is frequently overlooked. Both [5] and [3] propose simple heuristics:

1. For the input (first) layer, repeated occurrence of a signature is often taken as sufficient evidence, since first-layer signatures typically appear more frequently and the attacker commonly knows (or assumes) the number of signatures in that layer.
2. For deeper layers, partial control of the layer input produces partial signatures; in this case two signatures that collide (i.e., whose coefficients agree up to a fixed threshold) are treated as evidence that they originate from the same layer.

While the second criterion is reasonable in theory, we empirically found that selecting the most frequent signatures for the first layer often fails in practice, especially with smooth non-linear rectifiers. Backtracking can recover from such mistakes but is costly. We therefore propose a practical two-pronged strategy based on (i) filtering buckets and (ii) a lightweight oracle test.

Filtering buckets. A primary practical difficulty is the proliferation of false positives. In ReLU networks spurious signatures typically correspond to neurons in other layers, but our attack can also produce purely noisy candidates. To mitigate this, our implementation aggregates nearly identical signatures into buckets and retains only buckets that contain multiple collisions. Pure noise rarely repeats, so this simple frequency-based filtering greatly reduces false positives and effectively reduces the problem to a ReLU-like setting. To scale this approach we index signatures using Signed Random Projections [6], which avoids expensive all-pairs correlations.

The straight-line oracle. The input layer is special: a hyperplane from the first layer corresponds exactly to a neuron signature (it is not a linear combination of other neurons). As observed in [5], such an input-layer hyperplane remains “straight” when it crosses other hyperplanes; consequently, a point that satisfies the neuron’s affine equation will, with high probability, be detected by our attack. By contrast, a candidate signature originating from a deeper layer need not be valid in every neighborhood where its affine equation holds, so sampling a point that satisfies the equation may or may not trigger detection.

Concretely, consider the situations depicted in Figure 4:

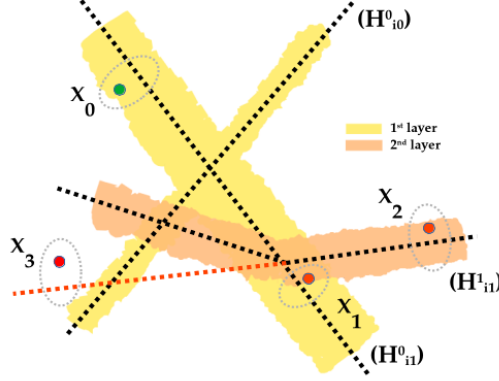


Fig. 4: Sketch of the first-layer detection idea.

1. $H^0_{i_1}$ is a first-layer hyperplane. Point x_0 satisfies the hyperplane equation and correctly detects the neuron (situation 1). Point x_1 also satisfies the equation but lies too close to another hyperplane $H^1_{i_1}$ and fails to detect it (situation 2).
2. $H^1_{i_1}$ is a hyperplane of an upper layer. Point x_2 detects it (situation 3), whereas x_3 does not because the affine relation defining $H^1_{i_1}$ is not valid in the neighborhood of x_3 (situation 4).

Because situations of type 1 and 4 are the most likely, simple random sampling of enough points that satisfy the candidate affine equation is usually sufficient to distinguish input-layer signatures from deeper-layer artifacts as illustrated by our practical results.

4.3 Experiments

To experimentally validate the claims made in this paper, we conducted several experiments. Our first set of experiments focused on small neural networks, with the goal of verifying whether the secret parameters of the first layer can indeed be recovered using only the *attraction* to critical hyperplanes and the exploration of adjacent linear regions, as predicted by our analysis. The corresponding results are summarized in Table 2 and clearly confirm our theoretical findings.

We then considered larger neural networks, with up to 100 000 secret parameters in the first layer. The corresponding results are reported in Table 3. All experiments in this setting were conducted with a time limit of 60 minutes.

As can be observed, the overall complexity appears to depend mainly on the number of secret parameters per neuron rather than on the total number of neurons. However, some neurons seem unreachable with our current approach. For instance, we let the experiment corresponding to MNIST (III) run for several hours without being able to recover any parameter of the missing neuron. Interestingly, this neuron has a significantly larger norm than the others, and we

Table 2: Detailed performance and recovery ratio for small neural networks. The time complexity is broken down into its main components: initial sieving (“Sieve”), adjacent linear zone technique (“ALZ”) for denoising, and checking that the neuron does belong to the first layer (“Check”). Note that the percentages sum up to a little less than 100%: the difference comes from other parts of the algorithm, which are cheap in comparison but not completely free.

Network	Time Complexity				Recovery Ratio		Data
	Total	Sieve	ALZ	Check	Neurons	Params	
$4 \times 4^4 \times 4$	1.3s	15.8%	62.1%	19.1%	4/4 (100%)	20/20 (100%)	$2^{15.27}$
$6 \times 5^3 \times 4$	1.6s	5.5%	85.7%	7.7%	5/5 (100%)	35/35 (100%)	$2^{15.78}$
$6 \times 6^5 \times 4$	2.9s	3.2%	90.2%	5.9%	6/6 (100%)	42/42 (100%)	$2^{16.57}$
$6 \times 7^3 \times 1$	2.3s	5.1%	83.5%	9.8%	7/7 (100%)	49/49 (100%)	$2^{16.25}$
$8 \times 8^8 \times 10$	6.2s	2.7%	92.8%	3.8%	8/8 (100%)	72/72 (100%)	$2^{17.67}$
$64 \times 8^8 \times 10$	7.7s	31.2%	38.9%	25.2%	8/8 (100%)	520/520 (100%)	$2^{17.39}$
$16 \times 16^8 \times 10$	42s	7.7%	88.1%	1.5%	16/16 (100%)	272/272 (100%)	$2^{20.14}$

believe that this property reduces the probability of being attracted to it during the sieving phase.

As a consequence, further investigation is required to fully understand the limitations of the techniques developed in this paper. Nevertheless, the results clearly indicate that the dominant cost of our algorithm lies in checking whether the candidate neurons found by algorithm do belong to the first layer.

Then, we evaluated the same neural networks while varying the activation functions used in the subsequent layers, in order to assess whether this choice impacts the efficiency of our technique. The corresponding results are summarized in Table 4 and suggest that the activation functions in the following layers do not significantly affect the effectiveness of our approach.

We also investigated how the choice of the activation function affects the efficiency of our technique. To this end, we performed several experiments on the same neural networks, modifying only the activation function used in the first layer, under a time limit of one hour per experiment. The corresponding results are reported in Table 5 and show that, when a time limit is imposed, some activation functions perform worse than others. Part of the observed differences may be attributed to the fact that the SiLU activation is considerably more expensive to evaluate than the others. As such, the difference may be partially attributed to the time limit, which naturally favors faster-to-compute activation functions. However, this factor alone does not fully explain the results. We suspect that this behavior is related to the shape of the second derivative of SiLU, as illustrated in Figure 5. We leave for future work a more in-depth investigation of this phenomenon.

Table 3: Detailed recovery ratio for larger neural networks. A time limit of 60 minutes was enforced for every experiment. The time complexity is broken down following the same format as the previous table. For networks in this table, only the “Sieve” and “ALZ” components use a Cython backend, and can be meaningfully compared; the “Check” component uses a slower Python backend.

Network	Time Complexity			Recovery ratio	
	Sieve	ALZ	Check	Neurons	Params
$784 \times 128 \times 1(*)$	26.57%	15.2%	51.42%	70/128 (54.7%)	54950/100480 (54.7%)
$100 \times 200^3 \times 10(*)$	35.7%	42.0%	7.2%	172/200 (86%)	17372/20200 (86%)
MNIST (II) $784 \times 8^8 \times 1 (*)$	22.1%	0.6%	71.5%	8/8 (100%)	6280/6280 (100%)
MNIST (III) $784 \times 16^8 \times 1 (*)$	21.5%	1.3%	68.8%	15/16 (93.8%)	11775/12560 (93.8%)
CIFAR10 (I) $3072 \times 8^8 \times 1 (*)$	24.5%	1.2%	73.4%	8/8 (100%)	24584/24584 (100%)

Table 4: Recovery ratio for different activation function sets (N layers).

Network	GeLU, ReLU $\times (N - 1)$		GeLU $\times N$		{Ge, Se, Mi, Si} $\times N$	
	Neurons	Params	Neurons	Params	Neurons	Params
$64 \times 8^8 \times 10$	8/8 (100%)	520/520 (100%)	8/8 (100%)	520/520 (100%)	8/8 (100%)	520/520 (100%)
$16 \times 16^8 \times 10$	16/16 (100%)	272/272 (100%)	16/16 (100%)	272/272 (100%)	15/16 (93.8%)	255/272 (93.8%)

Finally, we also attempted to recover secret parameters from layers beyond the first one and the results are reported in Table 6. However, our current implementation is not well-suited for this task, and we were unable to reach layers deeper than the fourth, mainly due to numerical precision issues. In addition, for deeper layers, it appears to be increasingly difficult to extract all the parameters associated with a given neuron.

These observations are consistent with [11], a recent work by Ito *et al.*, in which the authors show that claims made in earlier literature regarding the ease of “peeling off” successive layers are overly optimistic. Nevertheless, the techniques introduced in [13] could potentially be leveraged to overcome these limitations and enable the recovery of parameters from deeper layers than those reached by our proof-of-concept implementation.

Table 5: Detailed recovery ratio for hybrid activation function sets $(1 + (N - 1)$ layers).

Network	GeLU, Re $\times(N - 1)$		SeLU, Re $\times(N - 1)$		SiLU, Re $\times(N - 1)$		Mish, Re $\times(N - 1)$	
	Neurons	Params	Neurons	Params	Neurons	Params	Neurons	Params
$64 \times 8^8 \times 10$	8/8 (100%)	520/520 (100%)	8/8 (100%)	520/520 (100%)	8/8 (100%)	520/520 (100%)	8/8 (100%)	520/520 (100%)
$16 \times 16^8 \times 10$	16/16 (100%)	272/272 (100%)	15/16 (93.8%)	255/272 (93.8%)	15/16 (93.8%)	255/272 (93.8%)	15/16 (93.8%)	255/272 (93.8%)
$100 \times 200^3 \times 10$ (*)	156/200 (78%)	15.8k/20.2k (78%)	132/200 (68.5%)	13.8k/20.2k (68.5%)	58/200 (29%)	5.9k/20.2k (29%)	99/200 (49.5%)	10k/20.2k (49.5%)
$784 \times 128 \times 1$ (*)	64/128 (50%)	50k/100k (50%)	53/128 (41.4%)	41.6k/100k (41.4%)	36/128 (28.1%)	28.3k/100k (28.1%)	49/128 (38.3%)	38.5k/100k (38.3%)
MNIST (II) $784 \times 8^8 \times 1$ (*)	8/8 (100%)	6280/6280 (100%)	8/8 (100%)	6280/6280 (100%)	8/8 (100%)	6280/6280 (100%)	8/8 (100%)	6280/6280 (100%)
CIFAR10 (I) $3072 \times 8^8 \times 1$ (*)	5/8 (62.5%)	15.4k/24.6k (62.5%)	4/8 (50%)	12.3k/24.6k (50%)	4/8 (50%)	12.3k/24.6k (50%)	5/8 (62.5%)	15.4k/24.6k (62.5%)

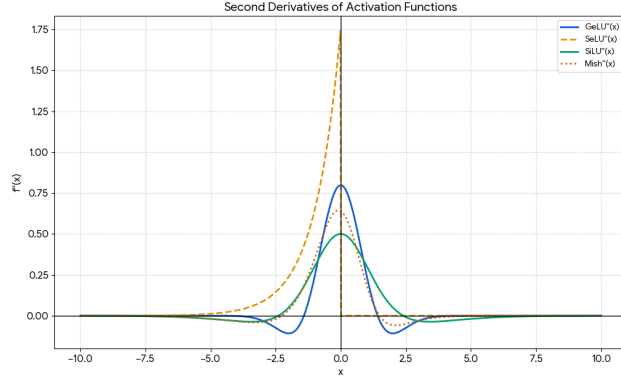


Fig. 5: Second-order derivatives of selected activation functions, illustrating the localized regions of non-linearity (transition bands) where the function’s curvature is (almost) non-zero.

5 Conclusion

In this paper, we introduced the first comprehensive cryptanalytic framework for extracting deep neural networks with non-linear activation functions beyond ReLU. By exploiting algebraic properties, higher-order derivatives, and geometric analysis of adjacent linear regions, our approach demonstrates that neither smoothness nor concealment of the activation functions substantially hinders extraction. In fact, certain activations make neuron signature recovery easier than

Table 6: Recovery ratio for deeper layers ($N > 0$) across different networks. For each network, the three quantities correspond respectively to the ratio of neurons detected in the target layer, the ratio of parameters recovered across all neurons in the layer, and the ratio of fully denoised parameters recovered.

Layer $N > 0$	Time Limit	$6 \times 6^5 \times 4$			$8 \times 8^8 \times 10$			$16 \times 16^8 \times 10$		
		Neurons det.	Weights part.	Weights recov.	Neurons det.	Weights part.	Weights recov.	Neurons det.	Weights part.	Weights recov.
2nd	8m	6/6 (100%)	31/36 (86.1%)	30/36 (83.3%)	8/8 (100%)	64/64 (100%)	64/64 (100%)	16/16 (100%)	256/256 (100%)	150/256 (58.6%)
3rd	1h	6/6 (100%)	32/36 (88.9%)	23/36 (63.9%)	8/8 (100%)	62/64 (96.9%)	56/64 (87.5%)	15/16 (93.8%)	205/256 (80.1%)	36/256 (14.1%)
4th	2h	5/6 (83.3%)	17/36 (47.22%)	15/36 (41.7%)	8/8 (100%)	53/64 (82.8%)	32/64 (50%)	13/16 (81.3%)	152/256 (59.38%)	0/256 (0%)

in the ReLU case. We also showed that activation functions can be identified post hoc, further reducing the security benefit of hiding architectural choices.

Our experiments confirm that these techniques are effective under both idealized high-precision and realistic double-precision environments, provided careful numerical calibration is applied. Together, these findings underscore that black-box access remains a powerful attack surface: DNNs, regardless of their activation functions, are vulnerable to efficient parameter recovery. This work therefore both generalizes the cryptanalytic understanding of neural model security and highlights the pressing need for principled defenses against model-stealing attacks.

References

1. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mane, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viegas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X.: Tensorflow: A system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). pp. 265–283. USENIX Association (2016)
2. Biryukov, A., Shamir, A.: Structural cryptanalysis of SASAS. *J. Cryptol.* **23**(4), 505–518 (2010). <https://doi.org/10.1007/S00145-010-9062-1>
3. Canales-Martínez, I.A., Chávez-Saab, J., Hambitzer, A., Rodríguez-Henríquez, F., Satpute, N., Shamir, A.: Polynomial time cryptanalytic extraction of neural network models. In: Proceedings of EUROCRYPT 2024. pp. 3–33. Springer (2024). https://doi.org/10.1007/978-3-031-58734-4_1
4. Carlini, N., Chávez-Saab, J., Hambitzer, A., Rodríguez-Henríquez, F., Shamir, A.: Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting. In: Proceedings of EUROCRYPT 2025. pp. 364–396. Springer (2025). https://doi.org/10.1007/978-3-031-91107-1_13

5. Carlini, N., Jagielski, M., Mironov, I.: Cryptanalytic extraction of neural network models. In: *Proceedings of CRYPTO 2020*. pp. 189–218. Springer (2020). https://doi.org/10.1007/978-3-030-56877-1_7
6. Charikar, M.S.: Similarity estimation techniques from rounding algorithms. In: *Proceedings of the 34th Annual ACM Symposium on Theory of Computing, STOC 2002*, Montréal, Québec, Canada, May 19–21, 2002. pp. 380–388. ACM (2002). <https://doi.org/10.1145/509907.509965>
7. Chen, Y., Dong, X., Guo, J., Shen, Y., Wang, A., Wang, X.: Hard-label cryptanalytic extraction of neural network models. In: *Proceedings of ASIACRYPT 2024*. pp. 207–236. Springer (2024). https://doi.org/10.1007/978-981-96-0944-4_7
8. Fefferman, C.: Reconstructing a neural net from its output. *Rev. Mat. Iberoam.* 10 **3**, 507—555 (1994). <https://doi.org/10.4171/RMI/160>
9. Foerster, H., Mullins, R.D., Shumailov, I., Hayes, J.: Beyond slow signs in high-fidelity model extraction. In: Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J.M., Zhang, C. (eds.) *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, Vancouver, BC, Canada, December 10 - 15, 2024 (2024), http://papers.nips.cc/paper_files/paper/2024/hash/22ae669a35bb9e70eb93ab77c1eff5b4-Abstract-Conference.html
10. Fornberg, B.: Generation of finite difference formulas on arbitrarily spaced grids. *Mathematics of Computation* **51**(184), 699–706 (1988). <https://doi.org/10.2307/2008770>
11. Ito, A., Miura, T., Todo, Y.: Is the hard-label cryptanalytic model extraction really polynomial? *Cryptology ePrint Archive*, Paper 2025/1868 (2025), <https://eprint.iacr.org/2025/1868>
12. Jagielski, M., Carlini, N., Berthelot, D., Kurakin, A., Papernot, N.: High accuracy and high fidelity extraction of neural networks. In: Capkun, S., Roesner, F. (eds.) *29th USENIX Security Symposium, USENIX Security 2020*, August 12–14, 2020. pp. 1345–1362. USENIX Association (2020), <https://www.usenix.org/conference/usenixsecurity20/presentation/jagielski>
13. Liu, H., Siproudhis, A., Experton, S., Lorenz, P., Boura, C., Peyrin, T.: Navigating the deep: Signature extraction on deep neural networks (2025), <https://arxiv.org/abs/2506.17047>
14. Lowd, D., Meek, C.: Adversarial learning. In: Grossman, R., Bayardo, R.J., Bennett, K.P. (eds.) *Proceedings of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Chicago, Illinois, USA, August 21–24, 2005. pp. 641–647. ACM (2005). <https://doi.org/10.1145/1081870.1081950>
15. Minaud, B., Derbez, P., Fouque, P., Karpman, P.: Key-recovery attacks on ASASA. *J. Cryptol.* **31**(3), 845–884 (2018). <https://doi.org/10.1007/S00145-017-9272-X>
16. mpmath: a Python library for arbitrary-precision floating-point arithmetic (version 1.3.0) (2023), <http://mpmath.org/>
17. Richardson, L.F.: The approximate arithmetical solution by finite differences of physical problems involving differential equations, with an application to the stresses in a masonry dam. *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character* **210**, 307–357 (1910). <https://doi.org/10.1098/rsta.1910.0009>

A Alternate solution to identifying the affine constants

We now describe a trivial alternative technique to retrieve the affine constants for an entire class of active functions. If an attacker can precisely locate a point belonging to an hyperplane then it instantly recovers the affine constant up to some multiplicative factor since the points' coordinates satisfy the equation. To locate such a point the attacker can use the zero of some derivative curve if there exist one being odd. GELU is such a function having all its $(2k + 1)$ -th (with $k > 0$) derivatives odd. However the computation of the third derivative is even less precise in general than the computation of a 2nd derivative. For this reason it is better to look for the extremum of the 2nd derivative but without the knowledge of the exact location of the hyperplane it is not a trivial task to find it as too many false positives are likely to occur. For this reason and being given the very specific nature of GELU, we implemented the following methodology:

1. The first step is to locate an approximate range around the zero of the 3rd, 5th and 7th derivative of the function. Higher order derivatives tend to fluctuate a lot but a zone where the zero of these 3 curve collide is likely to stand out even when the precision is low (see Figure 6a).
2. The attacker can then zoom in on the local extremum of the 2nd derivative within the range and repeat the process to increase the precision of the measurement. Initially the extremum is easy to locate (Figure 6a) but as the attacker reduces the range of research, the information becomes harder to retrieve (Figure 6c) and when the algorithm can no longer provide anymore satisfying answer (Figure 6d), the mean of the last interval is returned.

B Studying the impact of ϵ , $\|x_i\|$ and the choice of the derivation method to compute second order partial derivatives

In this appendix we report additional experiments on the empirical parameters that control the sieve and their impact on extraction success. Empirical evidence indicates two largely independent parameter groups merit separate study: the probing-vector norm $\|x_i\|$ (where x_i are randomly generated probing vectors) and the derivative-estimation parameters, namely the finite-difference step ϵ and the chosen stencil/formula.

Derivative parameters and method. Our test protocol is straightforward. We generated random test functions and computed high-precision reference values for their second-order partial derivatives at random points using `mpmath` [16]. For each sample we then computed the same derivatives using several double-precision (`float64`) finite-difference implementations based on Taylor-series stencils with 3, 4, 5 and 6 points, both with and without Richardson extrapolation; the two techniques can be combined. The raw results are summarized in Figure 7.

Two primary observations follow from these experiments:

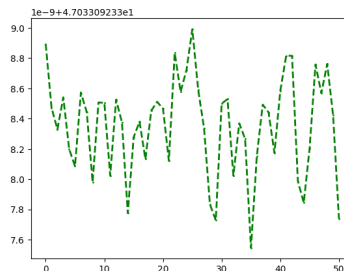
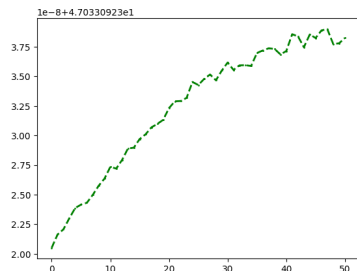
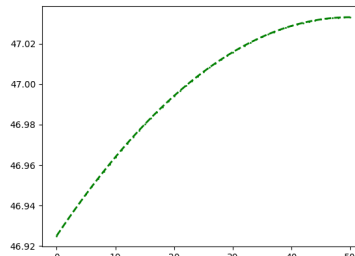
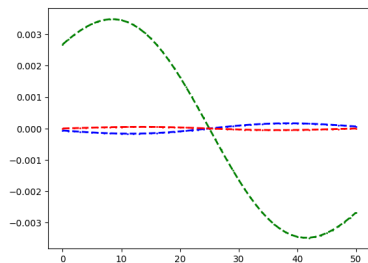


Fig. 6: An alternate technique to retrieve the affine constant.

- The best stand-alone accuracy (on our synthetic tests) is achieved with a 7-point stencil without Richardson extrapolation. In practice, however, Richardson seldom yields a net improvement here because it nearly doubles the number of function evaluations while providing little additional gain.
- The optimal finite-difference step is larger than one might expect on purely numerical grounds: the mean-optimal value found across our tests is on the order of 10^{-2} .

In realistic sieving scenarios the picture changes somewhat. We apply mixed partials, quotients, and other downstream operations, and the target function may be contaminated by nearby neurons that behave nonlinearly. Under these conditions we find no substantial advantage to the 7-point stencil compared to a 5-point stencil, and the latter is preferable for computational economy. More importantly, when measured in the full sieving pipeline, the step ϵ that maximizes practical success is closer to 10^{-1} (see Figure 8).

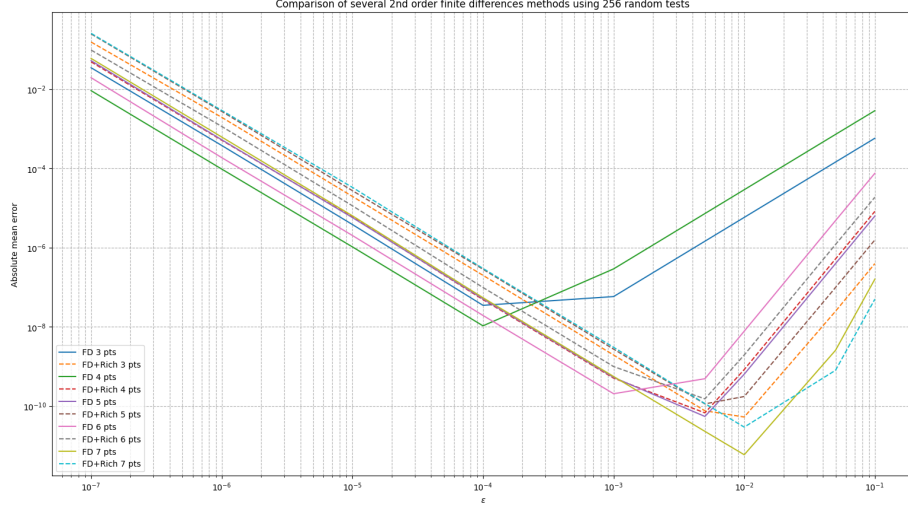


Fig. 7: Comparison of derivative accuracy for several finite-difference formulas as a function of the step ϵ and stencil size.

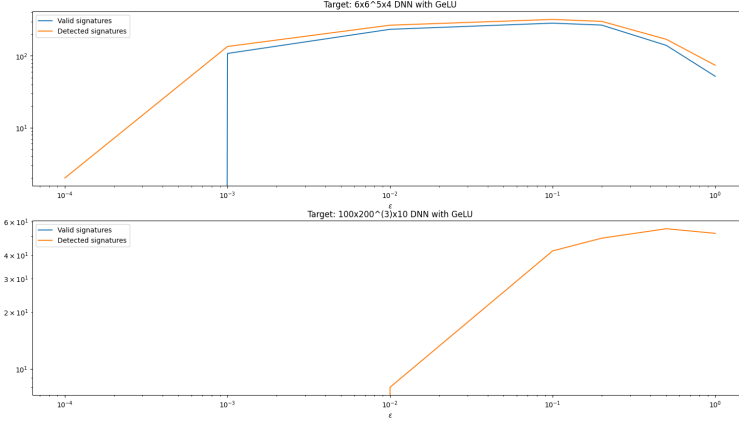


Fig. 8: Siever success rate as a function of ϵ for GELU activations. The activation choice has little effect on the optimal ϵ .

Notably, the optimal ϵ appears largely independent of the finite-difference formula and the activation function, but it may depend on properties of the target network (dimension, scale of weights), as suggested by Figure 8.

The norm $\|x_i\|$. While derivative tuning primarily affects numerical accuracy, the probing-vector norm $\|x_i\|$ controls the siever's ability to encounter inputs

dominated by a single neuron’s nonlinearity. To study this, we ran the sieve for a fixed number of iterations while varying $\|x_i\|$. For each run we counted (i) the number of confirmed signatures (a signature is confirmed when at least two extracted candidates are sufficiently correlated under a fixed threshold) and (ii) the total number of returned candidates (including false positives). These experiments used a 5-point Taylor formula with $\epsilon = 10^{-1}$ and are displayed in Figure 9. Additionally they were repeated for several activation functions such as GELU (Figure 10), SELU (Figure 11), the Sigmoid (Figure 12) and SiLU (Figure 13).

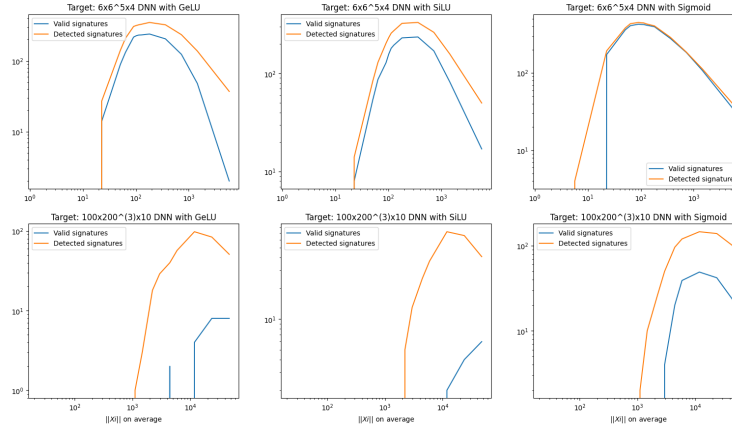


Fig. 9: Siever success rate versus probing norm $\|x_i\|$ for different activation functions (5-point finite differences, $\epsilon = 10^{-1}$).

Three conclusions follow:

1. There exists, on average, an optimal probing norm; this optimum tends to increase with the input dimension of the targeted layer but does not depend strongly on the activation function.
2. The activation function has little effect on the number of signatures collected for a given norm. Some activations (e.g., Sigmoid) produce cleaner, less noisy signatures and are therefore easier to classify, but the overall yield is governed primarily by $\|x_i\|$ and network geometry.
3. The threshold α has a significant impact by definition on the quantity of signatures returned by the sieve but has no impact on the choice of the norm.

Practically, we recommend a short bootstrap calibration phase prior to large-scale sieving: sweep ϵ and $\|x_i\|$ over coarse grids on the target network to find near-optimal values, then lock those parameters for the main extraction run.

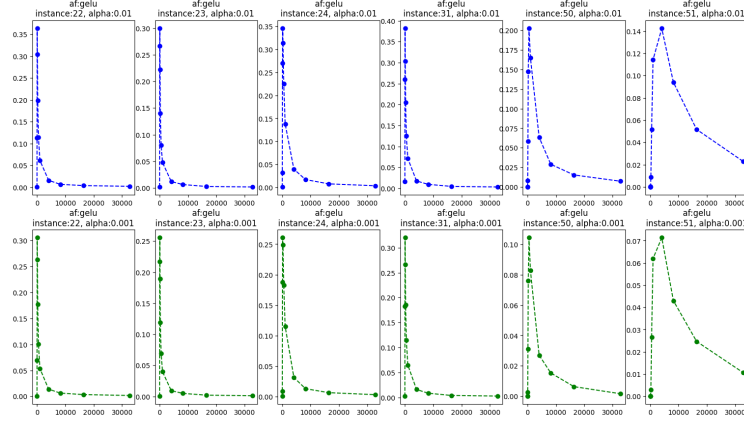


Fig. 10: Siever success rate versus probing norm $\|x_i\|$ for GELU and two different values of α (10^{-2} , 10^{-3}).

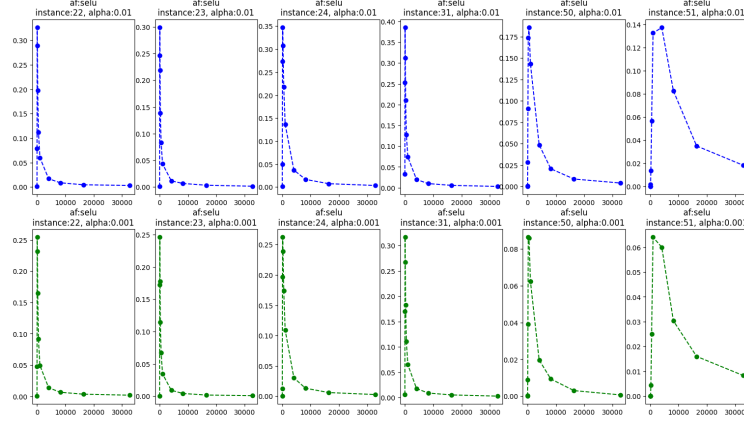


Fig. 11: Siever success rate versus probing norm $\|x_i\|$ for SELU and two different values of α (10^{-2} , 10^{-3}).

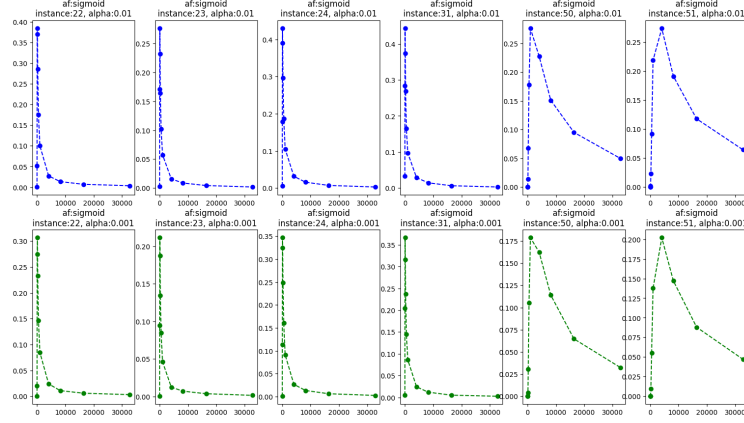


Fig. 12: Siever success rate versus probing norm $\|x_i\|$ for the Sigmoid and two different values of α (10^{-2} , 10^{-3}).

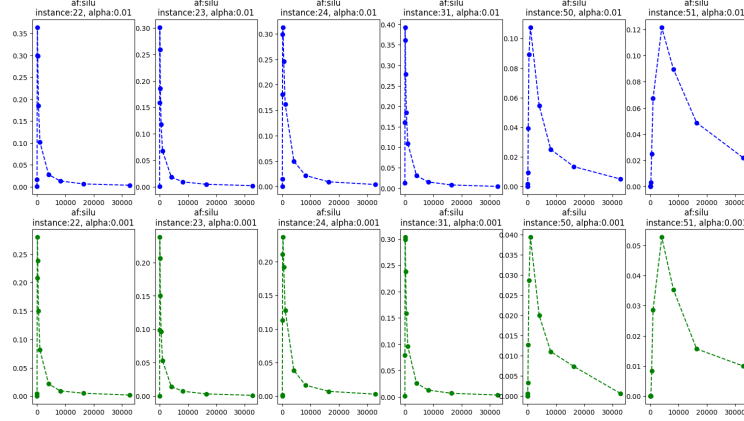


Fig. 13: Siever success rate versus probing norm $\|x_i\|$ for SiLU and two different values of α (10^{-2} , 10^{-3}).